
Action Dependency Graphs for Globally Optimal Coordinated Reinforcement Learning

Jianglin Ding, Jingcheng Tang, Gangshan Jing*

School of Automation, Chongqing University

20191921@cqu.edu.cn

tangjingcheng@stu.cqu.edu.cn

jinggangshan@cqu.edu.cn

Abstract

Action-dependent individual policies, which incorporate both environmental states and the actions of other agents in decision-making, have emerged as a promising paradigm for achieving global optimality in multi-agent reinforcement learning (MARL). However, the existing literature often adopts auto-regressive action-dependent policies, where each agent’s policy depends on the actions of all preceding agents. This formulation incurs substantial computational complexity as the number of agents increases, thereby limiting scalability. In this work, we consider a more generalized class of action-dependent policies, which do not necessarily follow the auto-regressive form. We propose to use the ‘action dependency graph (ADG)’ to model the inter-agent action dependencies. Within the context of MARL problems structured by coordination graphs, we prove that an action-dependent policy with a sparse ADG can achieve global optimality, provided the ADG satisfies specific conditions specified by the coordination graph. Building on this theoretical foundation, we develop a tabular policy iteration algorithm with guaranteed global optimality. Furthermore, we integrate our framework into several SOTA algorithms and conduct experiments in complex environments. The empirical results affirm the robustness and applicability of our approach in more general scenarios, underscoring its potential for broader MARL challenges.

1 Introduction

Achieving effective multi-agent reinforcement learning (MARL) in fully cooperative environments requires agents to coordinate their actions to maximize collective performance. Most existing MARL methods rely on independent policies [1, 2], where each agent makes decisions based solely on its state or observation. Although computationally tractable and scalable, these completely decentralized policies are often suboptimal [3]. The primary limitation lies in their tendency to converge to one of many Nash equilibrium solutions [4, 5], which may not correspond to the globally optimal solution.

The emergence of action-dependent policies [3] offers a promising solution to this challenge. By incorporating the actions of other agents into an agent’s decision-making process, action-dependent policies enable more effective cooperation and achieve superior performance compared to independent policies. We introduce the action dependency graph (ADG), a directed acyclic graph, to represent the action dependencies required for agents to make decisions. Theoretical studies [6, 7] demonstrate that policies with auto-regressive forms, associated with fully dense ADGs—where replacing each directed edge with an undirected edge yields a complete graph—guarantee global optimality. However, fully dense ADGs pose substantial scalability issues, as they require a high degree of interdependence and coordination.

*Corresponding author.

Sparse ADGs, which involve fewer inter-agent dependencies, offer a more scalable alternative. This leads to a critical question: can action-dependent policies with sparse ADGs still guarantee global optimality? To answer this question, we build on the framework of coordinated reinforcement learning [8], where the cooperative relationship between agents is described by a coordination graph (CG). We find that global optimality can still be achieved using an action-dependent policy with a sparse ADG, provided that a specific relationship between the ADG and the CG is satisfied.

The contributions of this paper are summarized as follows. (i). We introduce a novel theoretical framework that bridges the gap between scalability and optimality of MARL. To the best of our knowledge, this is the first work that seamlessly integrates coordination graphs and action-dependent policies. (ii). We provide a theoretical demonstration of the advantages of action-dependent policies with sparse ADGs over independent policies. Specifically, we prove that for MARL problems characterized by sparse CGs, policies associated with sparse ADGs satisfying our proposed condition achieve global optimality. This work pioneers the analysis of policies with sparse ADGs, in contrast with prior studies [4, 6, 9], which focus exclusively on fully dense ADGs. (iii). We integrate our theoretical insights with state-of-the-art algorithms, and show their effectiveness in the scenarios of coordination plymatrix games, adaptive traffic signal control, and StarCraft II.

2 Related Work

Independent policy. The majority of the literature on MARL represents the joint policy as the Cartesian product of independent individual policies. Value-based methods such as IQL [10], VDN [11], QMIX [12], and QTRAN [13] employ local value functions that depend only on the state or observation of each agent. Similarly, policy-based methods such as MADDPG [14], COMA [15], MAAC [16], and MAPPO [17] directly adopt independent policies. These approaches often fail to achieve global optimality, as they are not able to cover all strategy modes [3].

Coordination graph. Some value-based methods [18–21] recognize that the limitation of independent policies is due to a game-theoretic pathology known as relative overgeneralization [22]. To mitigate this, they employ a higher-order value decomposition framework by introducing the coordination graph (CG) [8]. In this graph, the vertices represent agents, and the edges correspond to pairwise interactions between agents in the local value functions. While CGs improve cooperation by considering inter-agent dependencies, the resulting joint policy cannot be decomposed into individual policies. Consequently, decision-making algorithms still require intensive computation, such as Max-Plus [23] or Variable Elimination (VE) [24]. When the CG is dense, these computations may become prohibitively time consuming, making the policy difficult to execute in real time.

Action-dependent policy. In contrast to independent policies, action-dependent policies [9, 25–27] incorporate not only the state, but also the actions of other agents into an agent’s decision-making process. The action dependencies among agents can be represented by a directed acyclic graph, which we refer to as the action dependency graph (ADG). In some literature, the action-dependent policy is also referred to as Bayesian policy [7] or auto-regressive policy [3]. Moreover, the use of action-dependent policies can be viewed as a mechanism to leverage communications for enhancing cooperation [28, 29]. Some approaches [4, 6, 30] transform a multi-agent MDP into a single-agent MDP with a sequential structure, enabling each agent to consider the actions of all previously decided agents during decision-making. This transformation ensures the convergent joint policy to be globally optimal [6]. However, the fully dense ADG makes these methods computationally expensive and impractical for large-scale systems. For more general ADGs, existing theories can only guarantee convergence to a Nash equilibrium solution [7]. Currently, no theoretical evidence demonstrates the superiority of action-dependent policies with sparse dependency graphs over independent policies.

3 Preliminary

In this paper, we formulate the cooperative multi-agent reinforcement learning problem as a Markov game, defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$, where $\mathcal{N} = \{1, \dots, n\}$ denotes the set of agents, $\mathcal{S}$ represents the finite state space, $\mathcal{A} = \prod_{i=1}^n \mathcal{A}_i$ is the Cartesian product of the agents’ finite action spaces, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ specifies the Markovian transition model, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ defines the reward function, and $\gamma \in [0, 1)$ is the discount factor.

A deterministic joint policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ maps a state to a joint action, while a stochastic joint policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps a state space to a probability distribution over the joint action space. Typically, a joint policy is represented as the product of *independent policies* $\pi_i : \mathcal{S} \rightarrow \Delta(\mathcal{A}_i)$, such that $\pi(a|s) = \prod_{i=1}^n \pi_i(a_i|s)$. The state value function induced by a joint policy π is defined as:

$$V^\pi(s) := \mathbb{E}_{a^t \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s^t, a^t) \middle| s^0 = s \right], \quad (1)$$

and the corresponding state-action value function is defined as:

$$Q^\pi(s, a) := \mathbb{E}_{a^t \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s^t, a^t) \middle| s^0 = s, a^0 = a \right]. \quad (2)$$

Given any $V \in \mathcal{R}(\mathcal{S})$, where $\mathcal{R}(\mathcal{S})$ denotes the set of all real-valued functions $J : \mathcal{S} \rightarrow \mathbb{R}$, we define:

$$Q^V(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s'). \quad (3)$$

The Bellman operator $T_\pi : \mathcal{R}(\mathcal{S}) \rightarrow \mathcal{R}(\mathcal{S})$ and the Bellman optimal operator $T : \mathcal{R}(\mathcal{S}) \rightarrow \mathcal{R}(\mathcal{S})$ are defined as:

$$T_\pi V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) Q^V(s, a), \quad TV(s) = \max_{a \in \mathcal{A}} Q^V(s, a) \quad (4)$$

The value function V^π satisfies the Bellman equation $T_\pi V^\pi = V^\pi$, while the optimal value function V^* satisfies the Bellman optimal equation $TV^* = V^*$.

3.1 Coordination Graph

In many practical scenarios such as sensor networks [31], wind farms [32], mobile networks [33], etc., the value function Q can be approximated as the sum of local value functions, each depending on the states and actions of a subset of agents. A widely used approach to representing this decomposition is the use of the *coordination graph* (CG) [8], which captures the pairwise coordination relationships between agents. Formally, we define a CG as follows.

Definition 3.1 (Coordination Graph). An undirected graph² $G_c = (\mathcal{N}, \mathcal{E}_c)$ is a CG of a function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, if there exists a local value function $Q_{ij} : \mathcal{S} \times \mathcal{A}_i \times \mathcal{A}_j \rightarrow \mathbb{R}$ for every edge $(i, j) \in \mathcal{E}_c$, and a local value function $Q_i : \mathcal{S} \times \mathcal{A}_i \rightarrow \mathbb{R}$ for every vertex $i \in \mathcal{N}$, such that

$$Q(s, a) = \sum_{i \in \mathcal{N}} Q_i(s, a_i) + \sum_{(i, j) \in \mathcal{E}_c} Q_{ij}(s, a_i, a_j). \quad (5)$$

Remark 3.2. If G_c is a subgraph of G'_c , and G_c is a CG of Q , then G'_c is also a CG of Q . Therefore, multiple CGs may correspond to the same value function Q .

Without loss of generality, we assume that G_c is connected; otherwise, the problem can be decomposed into independent subproblems depending on the connected components of G_c . In a connected graph, each vertex is involved in an edge, allowing the local value functions associated with vertices to be merged into those local value functions associated with edges, yielding:

$$Q(s, a) = \sum_{(i, j) \in \mathcal{E}_c} Q_{ij}(s, a_i, a_j). \quad (6)$$

Figure 1 (a) shows a CG where the function Q can be decomposed as:

$$Q(s, a) = Q_{12}(s, a_1, a_2) + Q_{13}(s, a_1, a_3) + Q_{23}(s, a_2, a_3). \quad (7)$$

We further characterize a class of Markov games where the equation (6) holds exactly, providing a natural generalization of the concept of decomposable games [34]. Due to the space limitation, the details is provided in the appendix.

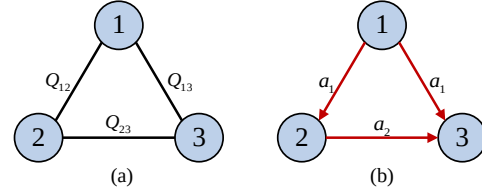


Figure 1: A coordination graph (a) and an action dependency graph (b).

²In this paper, the vertices and edges of the graph are represented by agent indices and index pairs.

3.2 Action Dependency Graph

In MARL, a joint policy is typically represented as the product of independent policies, each depending only on the current state. However, not every distribution over the joint action space can be represented as such a product. Given a distribution π over the joint action space, the chain rule of conditional probability states:

$$\pi(a|s) = \prod_{i=1}^n \pi_i(a_i|s, a_1, \dots, a_{i-1}). \quad (8)$$

This formulation gives rise to a broader class of individual policies, termed *action-dependent policies*, which are not necessarily conditionally independent but may instead depend on the actions of other agents. To formalize the action-dependent policy, we associate a joint policy π with a directed acyclic graph $G_d = (\mathcal{N}, \mathcal{E}_d)$, called the *action dependency graph* (ADG).

Definition 3.3 (Action Dependency Graph). A directed acyclic graph $G_d = (\mathcal{N}, \mathcal{E}_d)$ is said to be the ADG of a policy π if the policy of each agent i depends on the actions of agents in $N_d(i) = \{j \in \mathcal{N} : (j, i) \in \mathcal{E}_d\}$. More specifically, the joint policy π is expressed as:

$$\pi(a|s) = \prod_{i=1}^n \pi_i(a_i|s, a_{N_d(i)}), \quad (9)$$

where $\pi_i : \mathcal{S} \times \prod_{j \in N_d(i)} \mathcal{A}_j \rightarrow \Delta(\mathcal{A}_i)$ is the policy of agent i , and $a_{N_d(i)} = (a_j)_{j \in N_d(i)}$ ³ denotes the subvector of a indexed by $N_d(i)$.

Figure 1 (b) shows the ADG of a joint policy π with the following expression:

$$\pi(a|s) = \pi_1(a_1|s) \pi_2(a_2|s, a_1) \pi_3(a_3|s, a_1, a_2). \quad (10)$$

The acyclic nature of the ADG ensures that action dependencies do not form loops, which would otherwise lead to decision-making deadlocks, rendering policies infeasible. Every directed acyclic graph admits at least one topological sort, which can determine the order in which agents generate their actions. Without loss of generality, we assume that the indices of agents in $\mathcal{N}$ follow a topological sort. This implies that $j < i$ for all $j \in N_d(i)$ and all $i \in \mathcal{N}$.

Throughout this paper, we consider a Markov game structured by a CG. For analytical simplicity, we assume full observability, which means that each agent has access to the global state s . Nonetheless, our theoretical results can be integrated into practical algorithms, so as to adapt to scenarios with partial observability, which will be shown in Section 6.

4 ADG with Optimality Guarantee

While many methods designed for independent policies cannot guarantee convergence to a globally optimal policy, theoretical results do exist that ensure convergence to a Nash equilibrium policy [35, 36], which satisfies the following condition:

$$V^\pi(s) = \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s), \forall s \in \mathcal{S}, i \in \mathcal{N}. \quad (11)$$

A Nash equilibrium policy is equivalent to the agent-by-agent optimal policy introduced in [6] (see proof in Proposition E.1). Specifically, a policy π is agent-by-agent optimal if:

$$V^\pi(s) = \max_{\pi'_i} \mathbb{E}_{a_i \sim \pi'_i, a_{-i} \sim \pi_{-i}} [Q^\pi(s, a)], \forall s \in \mathcal{S}, i \in \mathcal{N}. \quad (12)$$

Here, $-S$ denotes the complement of set $S \subseteq \mathcal{N}$, and $-i$ is the shorthand for $-\{i\}$. The joint policy of agents in S is denoted by π_S . we also write joint policy π as (π_S, π_{-S}) . For brevity, we use $\mathbb{E}_\pi[\cdot]$ as the shorthand for $\mathbb{E}_{a \sim \pi}[\cdot]$, omitting the action when the context is clear.

Unlike independent policies, an action-dependent policy takes the actions of other agents (determined by G_d) as an additional part of the state representation. This enables the algorithms to achieve stronger solutions than agent-by-agent optimal policy, which we refer to as G_d -locally optimal policy.

³Throughout this paper, we use a vector with a set as its subscript to denote the components of the vector indexed by elements in that set.

Definition 4.1. A joint policy π is said to be G_d -locally optimal if for all $(s, a_{N_d(i)}) \in \mathcal{S} \times \prod_{j \in N_d(i)} \mathcal{A}_j$ and $i \in \mathcal{N}$, we have:

$$\mathbb{E}_{\pi_{-N_d(i)}} [Q^\pi(s, a)] = \max_{\pi'_i} \mathbb{E}_{\pi'_i, \pi_{-N_d(i)}} [Q^\pi(s, a)], \quad (13)$$

where $N_d[i] := N_d(i) \cup \{i\}$ denotes closed in-degree neighborhood of i in G_d .

Remark 4.2. When G_d is an empty graph, the definition of G_d -locally optimal is equivalent to that of agent-by-agent optimal. As the number of edges in G_d increases, the condition (13) becomes more restrictive. This results in fewer agent-by-agent optimal policies meeting the G_d -locally optimal criterion. In extreme, when G_d is a fully dense directed acyclic graph, the G_d -locally optimal policy aligns with the globally optimal policy. When the discussion is restricted to deterministic policy class, a G_d -locally optimal policy is always agent-by-agent optimal, though the converse does not necessarily hold. In this context, the class of G_d -locally optimal policies can be rigorously characterized as a subset of the agent-by-agent optimal policy class.

4.1 Coordination Polymatrix Game

To illustrate the suboptimality of Nash equilibrium and how action-dependent policies can address this issue, we consider an example of a *coordination polymatrix game* [37]. A coordination polymatrix game can be viewed as a single-step decision problem, defined by the Markov game tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle$, where $\mathcal{S} = \emptyset$ and $\gamma = 0$. Additionally, the game includes an undirected graph $G_c = (\mathcal{N}, \mathcal{E}_c)$ and a set of payoff functions $\{r_{ij}\}_{(i,j) \in \mathcal{E}_c}$, which determine the global reward $r(a) = \sum_{(i,j) \in \mathcal{E}_c} r_{ij}(a_i, a_j)$. In this context, r is equivalent to the (state)-action value function $Q : \mathcal{A} \rightarrow \mathbb{R}$, and G_c is the CG of Q . Figure 2 depicts a polymatrix game with three agents, each having two actions, $\mathcal{A}_i = \{0, 1\}$, $i = 1, 2, 3$. The payoff matrices specify the rewards for each pair of agents. For example, if agents 1 and 2 both choose action 0, they receive a payoff of 1.

When using deterministic independent policies, the joint policies $\pi = (1, 1, 1)$ and $\pi = (0, 0, 0)$ are both agent-by-agent optimal. However, only $\pi = (0, 0, 0)$ is globally optimal. Although $\pi = (1, 1, 1)$ is suboptimal, agents lack sufficient motivation to unilaterally change their actions, highlighting the limitations of independent policies in ensuring optimal cooperation.

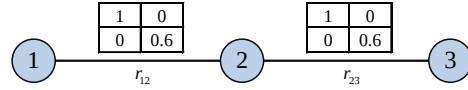


Figure 2: A polymatrix game on a line CG.

Now consider deterministic action-dependent policies associated with an ADG G_d , where $\mathcal{E}_d = \{(1, 2), (2, 3)\}$. When $\pi_1 = 1, \pi_2(0) = 0, \pi_2(1) = 1, \pi_3(0) = 0, \pi_3(1) = 1$, the joint policy remains $\pi = (1, 1, 1)$. If agent 1 switches to action 0, agents 2 and 3 will also switch to action 0, resulting in $r(0, 0, 0) = 2$, which exceeds $r(1, 1, 1) = 1.2$. This incentivizes agent 1 to choose action 0, leading to the globally optimal policy, which is also the only G_d -locally optimal policy in this case. However, not all ADGs guarantee that a G_d -locally optimal policy is globally optimal. For instance, if G_d has an edge set $\mathcal{E}_d = \{(1, 2), (3, 2)\}$. When $\pi_1 = 1, \pi_3 = 1, \pi_2(0, 0) = \pi_2(1, 0) = \pi_2(0, 1) = 0, \pi_2(1, 1) = 1$, the joint policy $\pi = (1, 1, 1)$ is G_d -locally optimal but not globally optimal.

4.2 Optimality Guarantee

There are some common criteria to determine whether a local optimum is also globally optimal, such as convexity of continuous functions or complete unimodality of discrete functions [38]. However, verifying these conditions often requires prior knowledge of the problem model, which is typically impractical. In this section, we introduce a novel condition for ensuring the global optimality of G_d -locally optimal policies, which is associated with the relationship between CG and ADG. The verification of this condition requires the network structure only.

Before presenting the results, we introduce some notations. Let $N_c(i)$ denote the neighbors of i in G_c . When i is replaced with a set S , $N_c(S)$ represents the neighbors of all agents in S , i.e., $N_c(S) = (\cup_{i \in S} N_c(i)) \setminus S$. Let $i^{[+]} = \{i, i+1, \dots, n\}$ represent the set of agents with indices not less than i .

Theorem 4.3 (Optimality of ADG, Proof in Appendix B). *Let policy π associated with ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$ be G_d -locally optimal. Let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be a CG of state-action value Q^π . If:*

$$N_d(i) = N_c(i^{[+]}) , \forall i \in \mathcal{N}, \quad (14)$$

then π is globally optimal, meaning $V^\pi = V^$.*

This theorem indicates that the ADG can be designed based on the CG, thereby ensuring that a G_d -locally optimal policy is also globally optimal.

Remark 4.4. Theorem 4.3 contains results in the literature as special cases. In particular, when both G_c and G_d are empty graphs, the Q -function can be decomposed in the form of VDN. Consequently, any Nash equilibrium policy is globally optimal, aligning with the findings in [34]. On the other hand, when G_c is a complete graph and G_d be a fully dense graph with edge set $\mathcal{E}_d = \{(i, j) \in \mathcal{N} \times \mathcal{N} : i < j\}$, the condition $N_d(i) = N_c(i^{[+]})$ is also satisfied. Thus, a G_d -locally optimal policy is guaranteed to be globally optimal for any CG, since every CG is a subgraph of a complete graph, aligning with the findings in [7].

While Theorem 4.3 requires the availability of a CG, which may seem restrictive, this is reasonable in many practical scenarios where the CG can be predefined. Moreover, even in domains without an apparent network structure, an approximate CG can still be learned [19, 20].

4.3 Finding Sparse ADGs

Although a fully dense ADG G_d ensures that any G_d -locally optimal policy achieves global optimality, training such policies can be computationally costly, limiting scalability. Thus, for sparse CGs, identifying sparse ADGs that satisfy (14) is essential to enhance efficiency.

If the indices of the agents are predetermined, the ADG can be uniquely determined by the condition (14). However, the choice of index order (topological sort) significantly affects the sparsity of the ADG, as illustrated in Figure 3. Determining the optimal index order is analogous to finding the optimal elimination order in VE algorithm, a problem known to be NP-complete [39]. Despite this complexity, practical heuristics can be employed, such as the greedy algorithm described in Appendix F Algorithm 2. The ADGs generated by the greedy algorithm for some simple network topologies are illustrated in Figure 4.

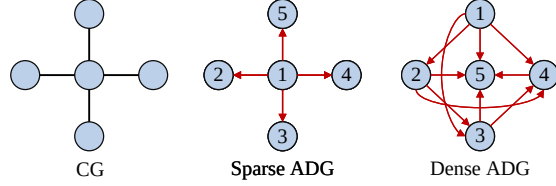


Figure 3: Different index orders of agents result in different sparsity of the ADG.

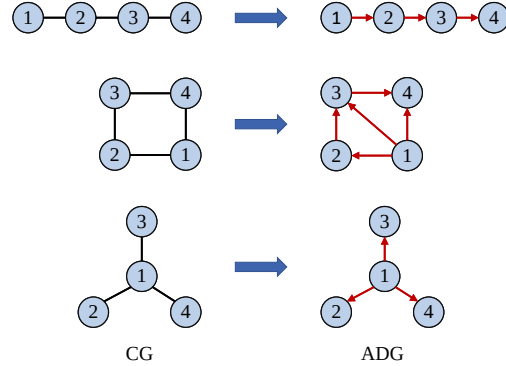


Figure 4: ADGs generated by Algorithm 2 for CG topologies: line, ring, and star.

5 Action-Dependent Policy Iteration

In this section, we introduce a policy iteration algorithm for MARL in a tabular setting. This algorithm elucidates the advantage of employing action-dependent policies, enabling convergence to a G_d -locally optimal policy rather than merely an agent-by-agent optimal policy. Under the conditions established in Theorem 4.3, it can be further concluded that our proposed policy iteration algorithm achieves convergence to the globally optimal solution.

Our approach extends the multi-agent policy iteration (MPI) framework proposed in [6], which decomposes the joint policy iteration step of standard policy iteration (PI) [40] into sequential updates of individual agents policies, thereby mitigating the computational complexity of PI. However, MPI only ensures convergence to an agent-by-agent optimal policy, which is frequently suboptimal. In

order to ensure the convergence of the joint policy to a G_d -locally optimal policy, we propose the Algorithm 1, which integrates deterministic action-dependent policies into the MPI framework.

Algorithm 1 Action-Dependent Multi-Agent Policy Iteration

```

Initialize policy  $\pi_i^0, i \in \mathcal{N}$  associated with the ADG  $G_d$ 
for  $k = 0, 1, \dots$  do
  // Policy Evaluation
  Solve for  $V^{\pi^k}$  using the Bellman equation  $V = T_{\pi^k} V$ 
  Calculate  $Q^{\pi^k}$  using  $V^{\pi^k}$ 
  // Policy Iteration
  for  $i = 1, 2, \dots, n$  do
    Denote the joint policy of  $(\pi_1^{k+1}, \dots, \pi_{i-1}^{k+1}, \pi_i^k, \dots, \pi_n^k)$  by  $\pi^{k,i}$ 
    Update policy:  $\pi_i^{k+1}(s, a_{N_d(i)}) \leftarrow \arg \max_{a_i \in \mathcal{A}_i} Q^{\pi^k}(s, a_{N_d[i]}, \pi_{-N_d[i]}^{k,i}(s, a_{N_d[i]})), \forall s, a_{N_d(i)}$ 
  end for
end for

```

To prevent non-convergence caused by alternating policies with identical value functions, we introduce the Assumption 5.1. In practical implementations, this assumption can be relaxed by checking whether π_i^k already achieves the maximum before updating π_i^{k+1} . If π_i^k achieves the maximum, then π_i^{k+1} is set to π_i^k to ensure stability. The convergence of Algorithm 1 is established in Theorem 5.2.

Assumption 5.1. In Algorithm 1, the set $\arg \max_{a_i \in \mathcal{A}_i} Q^{\pi^k}(s, a_{N_d[i]}, \pi_{-N_d[i]}^{k,i}(s, a_{N_d[i]}))$ is a singleton set for all $k = 1, 2, \dots$ and $i \in \mathcal{N}$.

Theorem 5.2 (Convergence of Algorithm 1, proof in Appendix C). *Assume $G_c = (\mathcal{N}, \mathcal{E}_c)$ is a CG of state-action value Q^π for all policies π , and Assumption 5.1 holds. Let $\{\pi^k\}$ be a sequence of joint policies generated by Algorithm 1, associated with an ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$. If the condition (14) is satisfied, then $\{\pi^k\}$ converges to a globally optimal policy in finite terms.*

Remark 5.3. Theorem 5.2 assumes that G_c serves as the CG of Q^π across all policies. However, this condition can be relaxed to require only G_c to act as the CG of the value function of the converged policy. The detailed proof of this relaxation is provided in Theorem C.1. Additionally, in Algorithm 1, while the convergence of all individual policies ensures that the resulting joint policy is G_d -locally optimal, the convergence is not always guaranteed if (14) is not satisfied. A similar issue has been encountered in [7], and we leave it as our future research topic.

6 Practical Algorithms

In this section, we introduce how to integrate the ADG into existing SOTA algorithms. Unlike prior CG-based approaches [18, 19], which face problems in representing individual policies and rely on complex techniques such as Max-Plus to compute optimal Q values, action-dependent policies exploit the structure of CGs while retaining the flexibility of independent policies. This facilitates seamless and versatile integration with a broad range of SOTA algorithms, rendering our theoretical framework applicable to general scenarios encompassing continuous state-action spaces and partial observability.

6.1 Policy-Based Methods

Policy-based methods optimize parameterized policies directly. To leverage our framework, we transform the independent policy $\pi_{\theta_i}(a_i|s)$ into an action-dependent form $\pi_{\theta_i}(a_i|s, a_{N_d(i)})$. This necessitates a corresponding adjustment to the optimization objective to accommodate the action-dependent policy. For instance, consider MAPPO [17], where the original objective is:

$$\mathcal{L}(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta_{\text{old}}}} \left[\sum_{i=1}^n \min \left(r_{\theta_i}(a_i, s) A_{\pi_{\theta_{\text{old}}}}(s, a), \text{clip}(r_{\theta_i}(a_i, s), 1 \pm \varepsilon) A_{\pi_{\theta_{\text{old}}}}(s, a) \right) \right],$$

where $r_{\theta_i}(a_i, s) = \frac{\pi_{\theta_i}(a_i|s)}{\pi_{\theta_{i,\text{old}}}(a_i|s)}$, $\mathcal{D}$ denotes the distribution of replay buffer. To adapt this objective for action-dependent policies, we replace $r_{\theta_i}(a_i, s)$ with $r_{\theta_i}(a_i, s, a_{N_d(i)}) = \frac{\pi_{\theta_i}(a_i|s, a_{N_d(i)})}{\pi_{\theta_{i,\text{old}}}(a_i|s, a_{N_d(i)})}$.

6.2 Value-Based Methods

Value-based methods implicitly represent policies through individual value functions $Q_i(s, a_i)$. To incorporate ADGs, we reformulate these into an action-dependent form $Q_i(s, a_i, a_{N_d(i)})$, enabling the derivation of corresponding action-dependent policies.

For example, in QMIX [12], we enforce a monotonicity constraint on the mixing network, ensuring:

$$\frac{\partial Q(s, a)}{\partial Q_i(s, a_{N_d(i)}, a_i)} \geq 0, \quad \forall i \in \mathcal{N}, a \in \mathcal{A}, s \in \mathcal{S}. \quad (15)$$

To obtain the greedy action $\arg \max_a Q(s, a)$, we compute $\arg \max_{a_i} Q_i(s, a_i, a_{N_d(i)})$ sequentially for each agent $i = 1, \dots, n$, leveraging the structural dependencies encoded by the ADG.

7 Experiments

To validate the theoretical results, we conduct an experimental evaluation of Algorithm 2 on coordination polymatrix games across diverse topological configurations. We will compare the performance corresponding to three types of ADGs, namely sparse ADG (satisfying condition (14)), fully dense ADG and empty ADG. To further explore the practical applicability of our method, we integrate ADGs into practical MARL algorithms, and evaluate their performance in more complex scenarios, including adaptive traffic signal control (ATSC) and StarCraft II. Detailed experimental protocols and hyperparameter settings are provided in Appendix G.

7.1 Coordination Polymatrix Games

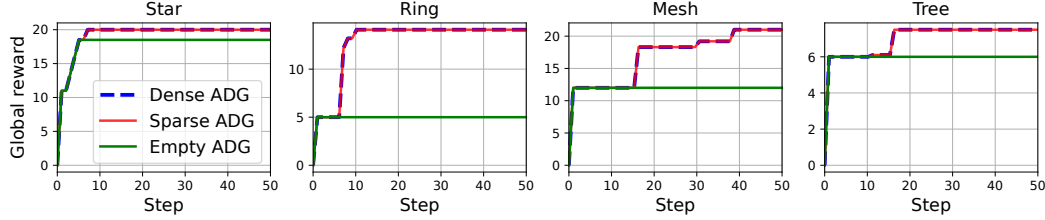


Figure 5: Results of coordination polymatrix game.

The experimental setup for the coordination polymatrix game is detailed in Section 4.1. We implement Algorithm 2 and evaluate its performance under various topological structures: star, ring, tree, and mesh. The specific CG topologies, corresponding sparse ADGs, and payoff matrices are elaborated in Appendix G. The payoff matrices are meticulously designed to induce multiple Nash equilibria, creating a challenging environment that tests the ability to achieve optimal cooperative outcomes.

Figure 5 demonstrates that both the sparse ADGs and the dense ADGs lead to globally optimal outcomes, while policies with empty ADGs frequently converge to suboptimal Nash equilibria. In fact, each learning step of the dense ADG takes more time than the sparse ADG since redundant action-dependencies are considered. These results underscore the critical role of a well-designed ADG in achieving global optimality with low computation costs.

7.2 ATSC

ATSC is a benchmark with a clear cooperation structure. We perform ATSC experiments using the Simulation of Urban Mobility (SUMO) platform [41], with the objective of optimizing traffic signal operations to enhance vehicular flow. Operating under partial observability, each of nine traffic signals accesses local intersection data in a 3x3 road network provided by SUMO-RL [42], with the reward defined as the negative aggregate pressure (difference between entering and exiting vehicles). The CG is constructed based on adjacency relationships between intersections, and the sparse ADG is derived using Algorithm 2.

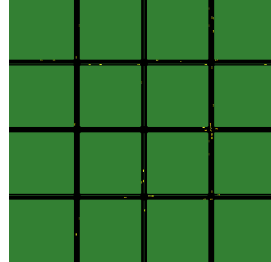


Figure 6: 3x3 road network.

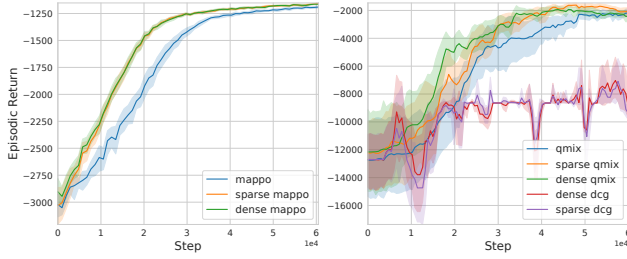


Figure 7: Results of ATSC.

advantage over empty ADGs, coupled with faster convergence. The limited performance gap may be attributed to the partial observability setting, which constrains agents’ ability to fully exploit action dependencies due to the restricted observational scope. Nevertheless, the consistent convergence behavior of sparse and dense ADGs corroborates the findings from the polymatrix game experiments. Conversely, DCG exhibits hyperparameter sensitivity and convergence difficulties under a restricted observational scope.

7.3 StarCraft II

We adopt the SMAC [43] benchmark to evaluate the performance of our method in complex environments. Recognizing the difficulty of inferring fixed CGs in StarCraft II, we introduce an artificially constructed CG and derive the corresponding ADG, as illustrated in Figure 1 in appendix. We train the QMIX algorithm augmented with ADG and compare its performance with the DCG algorithm, which leverages the same artificial CG. This comparison elucidates the relative strengths of ADG-based methods versus existing CG-based methods. We conduct ten experiments with distinct seeds on the super hard map MMM2, with results reported in Figure 9.



Figure 8: SMAC MMM2

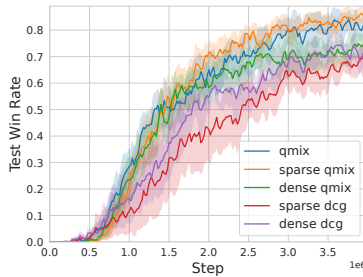


Figure 9: Results of SMAC.

The results reveal that QMIX with a sparse ADG outperforms the baseline QMIX algorithm, whereas QMIX with a dense ADG exhibits inferior performance. This suggests that dense ADG policies may require a greater volume of samples to achieve comparable performance. For DCG, the dense variant (using a complete CG) performs comparably to dense ADG, whereas the sparse variant (using the same CG as sparse ADG) underperforms. The suboptimal performance of sparse DCG may be attributable to the max-plus algorithm, which suffers from reduced accuracy as the number of CG edges decreases. In contrast, our ADG-based method circumvents reliance on the max-plus algorithm, mitigating this limitation.

8 Conclusion

In this paper, we introduced a novel theoretical framework for MARL with action-dependent policies and CGs. We rigorously proved that a G_d -locally optimal policy attains global optimality when the ADG meets CG-defined conditions under finite state and action spaces. Furthermore, by embedding our theory within the SOTA MARL algorithms, we provided empirical evidence on its effectiveness in practical scenarios. Recognizing that complex environments may involve unknown CGs, dynamic CGs, hypergraph CGs, we aim to explore the adaptability and potential of ADGs in these challenging settings in future research.

References

- [1] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of Reinforcement Learning and Control*, pages 321–384, 2021.

- [2] Afshin Oroojlooy and Davood Hajinezhad. A review of cooperative multi-agent deep reinforcement learning. *Applied Intelligence*, 53(11):13677–13722, 2023.
- [3] Wei Fu, Chao Yu, Zelai Xu, Jiaqi Yang, and Yi Wu. Revisiting some common practices in cooperative multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 6863–6877. PMLR, 2022.
- [4] Jianing Ye, Chenghao Li, Jianhao Wang, Qianchuan Zhao, and Chongjie Zhang. Towards global optimality in cooperative marl with sequential transformation. 2022.
- [5] Gangshan Jing, He Bai, Jemin George, Aranya Chakraborty, and Piyush K. Sharma. Distributed multiagent reinforcement learning based on graph-induced local value functions. *IEEE Transactions on Automatic Control*, 69(10):6636–6651, 2024.
- [6] Dimitri Bertsekas. Multiagent reinforcement learning: Rollout and policy iteration. *IEEE/CAA Journal of Automatica Sinica*, 8(2):249–272, 2021.
- [7] Dingyang Chen and Qi Zhang. Context-aware bayesian network actor-critic methods for cooperative multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 5327–5350. PMLR, 2023.
- [8] Carlos Guestrin, Michail G Lagoudakis, and Ronald Parr. Coordinated reinforcement learning. In *Proceedings of the 19th International Conference on Machine Learning*, pages 227–234, 2002.
- [9] Jiangxing Wang, Deheng Ye, and Zongqing Lu. More centralized training, still decentralized execution: Multi-agent conditional policy factorization. *arXiv preprint arXiv:2209.12681*, 2022.
- [10] Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of the 10th International Conference on Machine Learning*, pages 330–337, 1993.
- [11] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z Leibo, Karl Tuyls, et al. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, pages 2085–2087, 2018.
- [12] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 4295–4304. PMLR, 2018.
- [13] Kyunghwan Son, Daewoo Kim, Wan Ju Kang, David Earl Hostallero, and Yung Yi. Qtran: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 5887–5896. PMLR, 2019.
- [14] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems*, 30, 2017.
- [15] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [16] Shariq Iqbal and Fei Sha. Actor-attention-critic for multi-agent reinforcement learning. In *International Conference on Machine Learning*, pages 2961–2970. PMLR, 2019.
- [17] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35:24611–24624, 2022.
- [18] Wendelin Böhm, Vitaly Kurin, and Shimon Whiteson. Deep coordination graphs. In *International Conference on Machine Learning*, pages 980–991. PMLR, 2020.

- [19] Sheng Li, Jayesh K Gupta, Peter Morales, Ross Allen, and Mykel J Kochenderfer. Deep implicit coordination graphs for multi-agent reinforcement learning. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, pages 764–772, 2021.
- [20] Tonghan Wang, Liang Zeng, Weijun Dong, Qianlan Yang, Yang Yu, and Chongjie Zhang. Context-aware sparse deep coordination graphs. In *International Conference on Learning Representations*, 2022.
- [21] Jacopo Castellini, Frans A Oliehoek, Rahul Savani, and Shimon Whiteson. Analysing factorizations of action-value networks for cooperative multi-agent reinforcement learning. *Autonomous Agents and Multi-Agent Systems*, 35(2):25, 2021.
- [22] Liviu Panait, Sean Luke, and R Paul Wiegand. Biasing coevolutionary search for optimal multiagent behaviors. *IEEE Transactions on Evolutionary Computation*, 10(6):629–645, 2006.
- [23] Alex Rogers, Alessandro Farinelli, Ruben Stranders, and Nicholas R Jennings. Bounded approximate decentralised coordination via the max-sum algorithm. *Artificial Intelligence*, 175(2):730–759, 2011.
- [24] Umberto Bertele and Francesco Brioschi. *Nonserial dynamic programming*. Academic Press, Inc., 1972.
- [25] Jingqing Ruan, Yali Du, Xuantang Xiong, Dengpeng Xing, Xiyun Li, Linghui Meng, Haifeng Zhang, Jun Wang, and Bo Xu. Gcs: Graph-based coordination strategy for multi-agent reinforcement learning. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, pages 1128–1136, 2022.
- [26] Chuming Li, Jie Liu, Yinmin Zhang, Yuhong Wei, Yazhe Niu, Yaodong Yang, Yu Liu, and Wanli Ouyang. Ace: Cooperative multi-agent q-learning with bidirectional action-dependency. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 8536–8544, 2023.
- [27] Zhiyuan Li, Wenshuai Zhao, Lijun Wu, and Joni Pajarinen. Backpropagation through agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13718–13726, 2024.
- [28] Wei Duan, Jie Lu, and Junyu Xuan. Group-aware coordination graph for multi-agent reinforcement learning. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 3926–3934, 2024.
- [29] Yihe Zhou, Shunyu Liu, Yunpeng Qing, Kaixuan Chen, Tongya Zheng, Yanhao Huang, Jie Song, and Mingli Song. Is centralized training with decentralized execution framework centralized enough for marl? *arXiv preprint arXiv:2305.17352*, 2023.
- [30] Muning Wen, Jakub Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems*, 35:16509–16521, 2022.
- [31] Chongjie Zhang and Victor Lesser. Coordinated multi-agent reinforcement learning in networked distributed POMDPs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 25, pages 764–770, 2011.
- [32] Eugenio Bargiacchi, Timothy Verstraeten, Diederik Roijers, Ann Nowé, and Hado Hasselt. Learning to coordinate with coordination graphs in repeated single-stage multi-agent decision problems. In *International Conference on Machine Learning*, pages 482–490. PMLR, 2018.
- [33] Maxime Bouton, Hasan Farooq, Julien Forgeat, Shruti Bothe, Meral Shirazipour, and Per Karlsson. Coordinated reinforcement learning for optimizing mobile networks. *arXiv preprint arXiv:2109.15175*, 2021.
- [34] Zehao Dou, Jakub Grudzien Kuba, and Yaodong Yang. Understanding value decomposition algorithms in deep cooperative multi-agent reinforcement learning. *arXiv preprint arXiv:2202.04868*, 2022.

- [35] Runyu Zhang, Jincheng Mei, Bo Dai, Dale Schuurmans, and Na Li. On the global convergence rates of decentralized softmax gradient play in markov potential games. *Advances in Neural Information Processing Systems*, 35:1923–1935, 2022.
- [36] JG Kuba, R Chen, M Wen, Y Wen, F Sun, J Wang, and Y Yang. Trust region policy optimisation in multi-agent reinforcement learning. In *International Conference on Learning Representations*, page 1046, 2022.
- [37] Yang Cai and Constantinos Daskalakis. On minmax theorems for multiplayer games. In *Proceedings of the 20nd Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 217–234. SIAM, 2011.
- [38] Dario Bauso and Raffaele Pesenti. Team theory and person-by-person optimization with binary decisions. *SIAM Journal on Control and Optimization*, 50(5):3011–3028, 2012.
- [39] Jelle R Kok and Nikos Vlassis. Collaborative multiagent reinforcement learning by payoff propagation. *Journal of Machine Learning Research*, 7, 2006.
- [40] Richard S Sutton. Reinforcement learning: An introduction. *A Bradford Book*, 2018.
- [41] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using sumo. In *The 21st IEEE International Conference on Intelligent Transportation Systems*. IEEE, 2018.
- [42] Lucas N. Alegre. SUMO-RL. <https://github.com/LucasAlegre/sumo-rl>, 2019.
- [43] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The StarCraft Multi-Agent Challenge. *CoRR*, abs/1902.04043, 2019.
- [44] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018.
- [45] Tianshu Chu, Sandeep Chinchali, and Sachin Katti. Multi-agent reinforcement learning for networked system control. *arXiv preprint arXiv:2004.01339*, 2020.
- [46] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [47] Dimitri Bertsekas. *Abstract dynamic programming*. Athena Scientific, 2022.
- [48] Masako Bando, Katsuya Hasebe, Akihiro Nakayama, Akihiro Shibata, and Yuki Sugiyama. Dynamical model of traffic congestion and numerical simulation. *Physical Review E*, 51(2): 1035, 1995.
- [49] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [50] Jakub Grudzien Kuba, Xidong Feng, Shiyao Ding, Hao Dong, Jun Wang, and Yaodong Yang. Heterogeneous-agent mirror learning: A continuum of solutions to cooperative marl. *arXiv preprint arXiv:2208.01682*, 2022.
- [51] Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.

A Notations in Appendix

For clarity, we define the following notations:

- $i^- := \{1, 2, \dots, i-1\}$: the set of indices less than i .
- $i^+ := \{i+1, i+2, \dots, n\}$: the set of indices greater than i .
- $i^{[+]} := \{i, i+1, \dots, n\}$: the set of indices not less than i .
- $N_d[i] := N_d(i) \cup \{i\}$: the closed in-degree neighborhood of i .
- $N_c(S) := (\cup_{i \in S} N_c(i)) \setminus S$: the neighbors of all agents in S , and $N_d(S)$ is defined similarly.
- $\mathcal{E}_c[S_1, S_2]$: the subset of $\mathcal{E}_c$ containing edges between vertices in the sets S_1 and S_2 .
- $\mathcal{E}_c[S] := \mathcal{E}_c[S, S]$: the edges within the set S .
- $\mathcal{A}_S := \prod_{i \in S} \mathcal{A}_i$: the joint action space of agents in the set S .

B Proof of Optimality

To facilitate the proof of Theorem 4.3, we first propose four useful lemmas.

Lemma B.1. *Let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be an undirected graph and $G_d = (\mathcal{N}, \mathcal{E}_d)$ be a directed graph. If $N_d(i) \supseteq N_c(i^{[+]})$ and $j < i$ hold for any $j \in N_d(i)$ and $i \in \mathcal{N}$, then*

$$\mathcal{E}_c[k^{[+]}, k^-] = \mathcal{E}_c[k^{[+]}, N_d(k)], \forall k \in \mathcal{N}. \quad (16)$$

Proof. We first prove $\mathcal{E}_c[k^{[+]}, k^-] \supseteq \mathcal{E}_c[k^{[+]}, N_d(k)]$. It suffices to show $N_d(k) \subseteq k^-$, which can be obtained by noting that $j < k$ holds for any $j \in N_d(k)$ and $k \in \mathcal{N}$.

Next, we prove the reverse inclusion. Assume $i \in k^{[+]}$ and $(i, j) \in \mathcal{E}_c[k^{[+]}, k^-]$. By definition, $j \in N_c(k^{[+]})$. Since $N_d(k) \supseteq N_c(k^{[+]})$, it follows that $j \in N_d(k)$. Thus, $\mathcal{E}_c[k^{[+]}, k^-] \subseteq \mathcal{E}_c[k^{[+]}, N_d(k)]$. $\square$

Lemma B.2. *Let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be an undirected graph and $G_d = (\mathcal{N}, \mathcal{E}_d)$ be a directed graph. If $N_d(i) = N_c(i^{[+]})$ and $j < i$ hold for any $j \in N_d(i)$ and $i \in \mathcal{N}$, then*

$$N_d[k] \supseteq N_d(k^+), k = 1, 2, \dots, n-1. \quad (17)$$

Proof. We prove the statement by induction.

Base case: Show that the statement holds for $k = n-1$.

$$N_d[n-1] = (N_c((n-1)^{[+]}) \cup \{n-1\}) \supseteq N_c(n) = N_d(n) = N_d((n-1)^+). \quad (18)$$

Induction step: Assume the statement holds for $k+1$. We prove it for k .

$$N_d[k] = (N_c(k^{[+]}) \cup \{k\}) \supseteq N_c(k^+) = N_d(k+1). \quad (19)$$

By the induction hypothesis, we have:

$$N_d(k+1) = N_d[k+1] \setminus \{k+1\} \supseteq N_d((k+1)^+) \setminus \{k+1\}. \quad (20)$$

Therefore,

$$N_d[k] \supseteq N_d(k+1) \cup (N_d((k+1)^+) \setminus \{k+1\}) \supseteq N_d(k^+). \quad (21)$$

This completes the proof. $\square$

Lemma B.3. *Let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be a CG of Q , and $G_d = (\mathcal{N}, \mathcal{E}_d)$ be a directed graph. If $N_d(k) \supseteq N_c(k^{[+]})$ and $j < k$ hold for any $j \in N_d(k)$ and $k \in \mathcal{N}$, then given any i , there exist functions $Q_1 : \mathcal{S} \times \mathcal{A}_{N_d(i) \cup i^{[+]}} \rightarrow \mathbb{R}$ and $Q_2 : \mathcal{S} \times \mathcal{A}_{i^-} \rightarrow \mathbb{R}$ such that*

$$Q(s, a) = Q_1(s, a_{i^{[+]}}), a_{N_d(i)}) + Q_2(s, a_{i^-}), \quad \forall s \in \mathcal{S}, a \in \mathcal{A}. \quad (22)$$

Proof. Decomposing Q according to the structure of the CG G_c yields

$$Q(s, a) = \left(\sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, i^-]} + \sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[i^-, N_d(i)]} \right) Q_{jk}(s, a_j, a_k). \quad (23)$$

By Lemma B.1, $\mathcal{E}_c[i^{[+]}, i^-] = \mathcal{E}_c[i^{[+]}, N_d(i)]$. We rewrite the equation as:

$$Q(s, a) = \left(\sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[i^-, N_d(i)]} \right) Q_{jk}(s, a_j, a_k). \quad (24)$$

Define:

$$Q_1(s, a_{N_d(i)}, a_{i^{[+]}}) \triangleq \left(\sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[i^{[+]}, N_d(i)]} + \sum_{(j,k) \in \mathcal{E}_c[N_d(i)]} \right) Q_{jk}(s, a_j, a_k), \quad (25)$$

which is independent of $a_{i^- \setminus N_d(i)}$, and define:

$$Q_2(s, a_{i^-}) \triangleq \sum_{(j,k) \in \mathcal{E}_c[i^-, N_d(i)]} Q_{jk}(s, a_j, a_k), \quad (26)$$

which is independent of $a_{i^{[+]}}$. Then we have (22). $\square$

Lemma B.4. Let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be a CG of Q . Let $\pi_{-N_d[i]}$ be any subpolicy of a joint policy associated with an ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$. If $N_d(k) = N_c(k^{[+]})$ holds for any $k \in \mathcal{N}$, then we have:

$$\arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q(s, a'_{N_d(i)}, a_{-N_d(i)})] = \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{i^+}} [Q(s, a'_{i^-}, a_{i^{[+]}})], \quad \forall s \in \mathcal{S}, a'_{i^-} \in \mathcal{A}_{i^-}. \quad (27)$$

Proof. By Lemma B.2, the actions of the agents in $N_d(i^+)$ are a subset of the actions of agents in $N_d[i]$. That is, π_{i^+} is independent of $a_{i^- \setminus N_d(i)}$. Therefore, for any $s \in \mathcal{S}, a'_{i^-} \in \mathcal{A}_{i^-}$, it holds that

$$\mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q(s, a'_{i^-}, a_{i^{[+]}})] = \mathbb{E}_{\pi_i, \pi_{i^+}} [Q(s, a'_{i^-}, a_{i^{[+]}})]. \quad (28)$$

Now, consider:

$$\begin{aligned} & \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q(s, a'_{N_d(i)}, a_{-N_d(i)})] \\ &= \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q_1(s, a'_{N_d(i)}, a_{i^{[+]}}) + Q_2(s, a'_{N_d(i)}, a_{i^- \setminus N_d(i)})] \quad (\text{by Lemma B.3}) \\ &= \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q_1(s, a'_{N_d(i)}, a_{i^{[+]}}) + Q_2(s, a'_{i^-})] \quad (\text{since } Q_2 \text{ is independent of } a_i) \\ &= \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{-N_d[i]}} [Q(s, a'_{i^-}, a_{i^{[+]}})] \\ &= \arg \max_{\pi_i(\cdot | s, a'_{N_d(i)})} \mathbb{E}_{\pi_i, \pi_{i^+}} [Q(s, a'_{i^-}, a_{i^{[+]}})]. \quad (\text{by Equation (28)}) \end{aligned} \quad (29)$$

Thus, the proof is complete. $\square$

Proof of Theorem 4.3

We prove by induction that for every $i \in \mathcal{N}, a'_{i^-} \in \mathcal{A}_{i^-}$ and $s \in \mathcal{S}$:

$$\mathbb{E}_{\pi_{i^{[+]}}} [Q^\pi(s, a'_{i^-}, a_{i^{[+]}})] = \max_{a_{i^{[+]}}} Q^\pi(s, a'_{i^-}, a_{i^{[+]}}). \quad (30)$$

Base case: Show that the statement holds for index n .

By the definition of G_d -optimality and Lemma B.4, for any $s \in \mathcal{S}$, $a'_{n-} \in \mathcal{A}_{n-}$:

$$\begin{aligned} \pi_n(\cdot | s, a'_{N_d(n)}) &\in \arg \max_{\pi'_n(\cdot | s, a'_{N_d(n)})} \mathbb{E}_{\pi'_n, \pi_{-N_d[n]}} \left[Q^\pi(s, a_{-N_d[n]}, a'_{N_d(n)}, a_n) \right] \\ &= \arg \max_{\pi'_n(\cdot | s, a'_{N_d(n)})} \mathbb{E}_{\pi'_n} [Q^\pi(s, a'_{n-}, a_n)]. \end{aligned} \quad (31)$$

This implies:

$$\mathbb{E}_{\pi_n} [Q^\pi(s, a'_{n-}, a_n)] = \max_{\pi'_n(\cdot | s, a'_{N_d(n)})} \mathbb{E}_{\pi'_n} [Q^\pi(s, a'_{n-}, a_n)] = \max_{a_n} Q^\pi(s, a'_{n-}, a_n). \quad (32)$$

Induction step: Assume the induction hypothesis holds for $k+1$, we will show that it holds for k .

By the definition of G_d -optimality and Lemma B.4, for any $s \in \mathcal{S}$, $a'_{k-} \in \mathcal{A}_{k-}$.

$$\begin{aligned} \pi_k(\cdot | s, a'_{N_d(k)}) &\in \arg \max_{\pi'_k(\cdot | s, a'_{N_d(k)})} \mathbb{E}_{\pi'_k, \pi_{-N_d[k]}} \left[Q^\pi(s, a'_{N_d(k)}, a_{-N_d(k)}) \right] \\ &= \arg \max_{\pi'_k(\cdot | s, a'_{N_d(k)})} \mathbb{E}_{\pi'_k, \pi_{k+}} [Q^\pi(s, a'_{k-}, a_{k[+]})]. \end{aligned} \quad (33)$$

Thus, we have:

$$\begin{aligned} \mathbb{E}_{\pi_k, \pi_{k+}} [Q^\pi(s, a'_{k-}, a_{k[+]})] &= \max_{\pi'_k(\cdot | s, a'_{N_d(k)})} \mathbb{E}_{\pi'_k, \pi_{k+}} [Q^\pi(s, a'_{k-}, a_{k[+]})] \\ &= \max_{\pi'_k(\cdot | s, a'_{N_d(k)})} \mathbb{E}_{\pi'_k, \pi_{(k+1)[+]}} [Q^\pi(s, a'_{k-}, a_{k[+]})] \\ &= \max_{\pi'_k(\cdot | s, a'_{N_d(k)})} \mathbb{E}_{\pi'_k} \left[\max_{a_{(k+1)[+]}} Q^\pi(s, a'_{k-}, a_{k[+]}) \right] \quad (\text{by induction hypothesis}) \\ &= \max_{a_k} \max_{a_{(k+1)[+]}} Q^\pi(s, a'_{k-}, a_{k[+]}) \\ &= \max_{a_{k[+]}} Q^\pi(s, a'_{k-}, a_k). \end{aligned} \quad (34)$$

For $k = 1$, we have:

$$V^\pi(s) = \mathbb{E}_\pi [Q^\pi(s, a)] = \max_a Q^\pi(s, a) = TV^\pi(s). \quad (35)$$

which shows that V^π is a fixed point of the Bellman optimal operator T . Therefore, $V^\pi = V^*$, implying that π is globally optimal. $\square$

C Proof of Convergence

Before proving Theorem 5.2, we first propose a theorem showing the convergence relying on a more relaxed condition. Specifically, Theorem C.1 only requires that the state-action value functions of the converged policy satisfy the decomposed form of the CG, rather than requiring this property for all policies.

Theorem C.1. Assume Assumption 5.1 holds. Let $\{\pi^k\}$ be a sequence of joint policies generated by Algorithm 1, associated with an ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$. Let $V^\circ = \lim_{k \rightarrow \infty} V^{\pi^k}$, and let $G_c = (\mathcal{N}, \mathcal{E}_c)$ be a CG of state-action value Q^{V° . If:

$$N_d(i) = N_c(i^{[+]}), \forall i \in \mathcal{N}, \quad (36)$$

then $\{\pi^k\}$ converges to a globally optimal policy in finite terms.

The proof of Theorem C.1 proceeds in three steps. First, we establish the convergence of the joint policy, as shown in Lemma C.2. Next, we prove the convergence of individual agent policies. Finally, using Theorem 4.3, we guarantee the global optimality of the converged policy.

Lemma C.2. *If $\{\pi^k\}$ is a sequence of joint policies generated by Algorithm 1, associated with an ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$, then $\{V^{\pi^k}\}$ will converge to some V° in finite terms.*

Proof. According to Algorithm 1, for any $s \in \mathcal{S}$ and $k \geq 0$:

$$\begin{aligned}
T_{\pi^{k+1}} V^{\pi^k}(s) &= Q^{\pi^k}(s, \pi^{k+1}(s)) \\
&= \max_{a_n} Q^{\pi^k}(s, a_n, \pi_{-n}^{k,n}(s)) \\
&\geq Q^{\pi^k}(s, \pi^{k,n}(s)) \\
&= \max_{a_{n-1}} Q^{\pi^k}(s, a_{n-1}, \pi_{-(n-1)}^{k,n-1}(s, a_{n-1})) \\
&\geq Q^{\pi^k}(s, \pi^{k,n-1}(s)) \\
&\dots \\
&\geq Q^{\pi^k}(s, \pi^k(s)) \\
&= T_{\pi^k} V^{\pi^k}(s) = V^{\pi^k}(s).
\end{aligned} \tag{37}$$

Since $T_{\pi^{k+1}} V^{\pi^k}(s) \geq V^{\pi^k}(s)$, by the monotonicity of the Bellman operator [47], we have

$$V^{\pi^{k+1}}(s) = \lim_{m \rightarrow \infty} T_{\pi^{k+1}}^m V^{\pi^{k+1}}(s) \geq T_{\pi^{k+1}} V^{\pi^k}(s) \geq V^{\pi^k}(s). \tag{38}$$

Thus, $\{V^{\pi^k}\}$ is non-decreasing and bounded above by the optimal value function V^* . Therefore, $\{V^{\pi^k}\}$ converges to some V° . Further, since the number of deterministic policies is finite, $\{V^{\pi^k}\}$ will converge to V° in finite terms. $\square$

Proof of Theorem C.1

By Lemma C.2, $\{V^{\pi^m}\}$ converges to V° in finite terms. Assume $V^{\pi^m} = V^\circ$ for all $m \geq M$. We prove by induction that for any agent $i \in \mathcal{N}$ there exist a policy π_i° and $M_i > M$ such that $\pi_i^\circ = \pi_i^m, \forall m \geq M_i$, making π_i° the convergence point.

Base case: Show that the statement holds for agent n .

For any $s \in \mathcal{S}, a_{N_d(n)} \in \mathcal{A}_{N_d(n)}$, and $m \geq M_n > M$, we have:

$$\begin{aligned}
\pi_n^m(s, a_{N_d(n)}) &\in \arg \max_{a_n \in \mathcal{A}_n} Q^{V^{\pi^{m-1}}}(s, \pi_{-N_d[n]}^{m-1,n}(s, a_{N_d[n]}), a_{N_d(n)}, a_n) \\
&\stackrel{(a)}{=} \arg \max_{a_n \in \mathcal{A}_n} Q^{V^\circ}(s, \pi_{n-\setminus N_d(n)}^m(s, a_{N_d(n)}), a_{N_d(n)}, a_n) \\
&\stackrel{(b)}{=} \arg \max_{a_n \in \mathcal{A}_n} \left[Q_1^{V^\circ}(s, a_{N_d(n)}, a_n) + Q_2^{V^\circ}(s, \pi_{n-\setminus N_d(n)}^m(s, a_{N_d(n)}), a_{N_d(n)}) \right] \\
&\stackrel{(c)}{=} \arg \max_{a_n \in \mathcal{A}_n} Q_1^{V^\circ}(s, a_{N_d(n)}, a_n),
\end{aligned} \tag{39}$$

where

- (a): $\pi_{n-\setminus N_d(n)}^m$ depends at most on actions of agents before n due to the topological sort of G_d ;
- (b): follows from Lemma B.3;
- (c): $Q_2^{V^\circ}$ is independent of a_n .

By Assumption 5.1, we define $\pi_n^\circ := \pi_n^{M_n} = \pi_n^m, \forall m \geq M_n$.

Induction step: Assume the induction hypothesis holds for all agents $j > k$. We will show that it also holds for agent k .

For any $s \in \mathcal{S}$, $a_{N_d(k)} \in \mathcal{A}_{N_d(k)}$, and $m \geq M_k > \max_{j>k} M_j$, we have:

$$\begin{aligned}
\pi_k^m(s, a_{N_d(k)}) &\in \arg \max_{a_k \in \mathcal{A}_k} Q^{V^{\pi^{m-1}}} \left(s, \pi_{-N_d[k]}^{m-1,k}(s, a_{N_d[k]}), a_{N_d(k)}, a_k \right) \\
&\stackrel{(a)}{=} \arg \max_{a_k \in \mathcal{A}_k} Q^{V^\circ} \left(s, \pi_{k^- \setminus N_d(k)}^m(s, a_{N_d(k)}), a_{N_d(k)}, a_k, \pi_{k^+}^{m-1}(s, a_{N_d[k]}) \right) \\
&\stackrel{(b)}{=} \arg \max_{a_k \in \mathcal{A}_k} Q^{V^\circ} \left(s, \pi_{k^- \setminus N_d(k)}^m(s, a_{N_d(k)}), a_{N_d(k)}, a_k, \pi_{k^+}^\circ(s, a_{N_d[k]}) \right) \\
&\stackrel{(c)}{=} \arg \max_{a_k \in \mathcal{A}_k} \left[Q_1^{V^\circ}(s, a_{N_d(k)}, a_k, \pi_{k^+}^\circ(s, a_{N_d[k]})) + Q_2^{V^\circ}(s, a_{N_d(k)}, \pi_{k^- \setminus N_d(k)}^m(s, a_{N_d(k)})) \right] \\
&\stackrel{(d)}{=} \arg \max_{a_k \in \mathcal{A}_k} Q_1^{V^\circ}(s, a_{N_d(k)}, a_k, \pi_{k^+}^\circ(s, a_{N_d[k]})).
\end{aligned} \tag{40}$$

The four equalities are explained as follows.

- (a): from Lemma B.2, $a_{N_d(k^+)}$ is a part of $a_{N_d[k]}$. Thus, $\pi_{k^+}^{m-1}$ depends at most on the actions of the agents in $N_d[k]$. $\pi_{k^- \setminus N_d(k)}^m$ depends at most on the actions of the agents before k due to the topological sort of G_d ;
- (b): follows from the induction hypothesis;
- (c): follows from Lemma B.3;
- (d): $Q_2^{V^\circ}$ is independent of a_k .

By Assumption 5.1, we define $\pi_k^\circ := \pi_k^{M_k} = \pi_k^m, \forall m \geq M_k$.

Thus, $\{\pi_i^m\}$ converges to π_i° in finite terms, and $V^\circ = V^{\pi^\circ}$. Further, π_i° satisfies:

$$\pi_i^\circ(s, a_{N_d(i)}) = \arg \max_{a_i \in \mathcal{A}_i} Q^{\pi^\circ} \left(s, \pi_{-N_d[i]}^\circ(s, a_{N_d[i]}), a_{N_d(i)}, a_i \right). \tag{41}$$

When we view π_i° as a stochastic policy, it holds that,

$$\mathbb{E}_{\pi_{-N_d(i)}^\circ} \left[Q^{\pi^\circ}(s, a) \right] = \max_{a_i} \mathbb{E}_{\pi_{-N_d[i]}^\circ} \left[Q^{\pi^\circ}(s, a) \right] = \max_{\pi_i'} \mathbb{E}_{\pi_i', \pi_{-N_d[i]}^\circ} \left[Q^{\pi^\circ}(s, a) \right]. \tag{42}$$

Therefore, π° is a G_d -locally optimal policy. It follows from Theorem 4.3 that π° is globally optimal. $\square$

Proof of Theorem 5.2

This is a direct consequence of Theorem C.1.

By Lemma C.2, there exist V° and M such that $V^{\pi^m} = V^\circ$ for all $m \geq M$. Since G_c is a CG of Q^π for all policies π , it is also a CG of $Q^{V^\circ} = Q^{\pi^M}$. By Theorem C.1, $\{\pi^k\}$ converges to a globally optimal policy. $\square$

D A Class of Markov Games where CGs Exactly Exist

We characterize a class of Markov games where the equation (6) holds exactly. Specifically, we assume the reward function r and transition model P can be decomposed over the edges of G_c , ensuring that G_c serves as a CG for the value function Q^V .

Proposition D.1. *Let r and P denote the reward function and the transition model of a Markov game, respectively. Suppose there exist two sets of functions $\{r_{ij}\}_{(i,j) \in \mathcal{E}_c}$ and $\{P_{ij}\}_{(i,j) \in \mathcal{E}_c}$, such that*

$$r(s, a) = \sum_{(i,j) \in \mathcal{E}_c} r_{ij}(s, a_i, a_j) \tag{43}$$

and

$$P(s'|s, a) = \sum_{(i,j) \in \mathcal{E}_c} P_{ij}(s'|s, a_i, a_j), \tag{44}$$

where $\mathcal{E}_c$ is the edge set of $G_c = (\mathcal{N}, \mathcal{E}_c)$. Then G_c is a CG of Q^V for any $V \in \mathcal{R}(\mathcal{S})$.

Proof. For any real-valued function $V : \mathcal{S} \rightarrow \mathbb{R}$, it holds that

$$\begin{aligned}
Q^V(s, a) &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V(s') \\
&= \sum_{(i,j) \in \mathcal{E}_c} r_{ij}(s, a_i, a_j) + \gamma \sum_{s' \in \mathcal{S}} \sum_{(i,j) \in \mathcal{E}_c} P_{ij}(s'|s, a_i, a_j) V(s') \\
&= \sum_{(i,j) \in \mathcal{E}_c} \left(r_{ij}(s, a_i, a_j) + \gamma \sum_{s' \in \mathcal{S}} P_{ij}(s'|s, a_i, a_j) V(s') \right) \\
&\triangleq \sum_{(i,j) \in \mathcal{E}_c} Q_{ij}^V(s, a_i, a_j).
\end{aligned} \tag{45}$$

Therefore, G_c is a CG of Q^V . $\square$

E Proof of the Equivalence Between Nash Equilibrium and Agent-by-Agent Optimal

We show the equivalence between the Nash equilibrium defined in (11) and the agent-by-agent optimal defined in (12) by proposing a proposition as follows.

Proposition E.1. *Let π be a joint policy of independent policies $\pi_i, i \in \mathcal{N}$. Then, The following claims are equivalent:*

- (i) $\mathbb{E}_{s \sim \mu} [V^\pi(s)] = \max_{\pi'_i} \mathbb{E}_{s \sim \mu} [V^{\pi'_i, \pi_{-i}}(s)], \mu(s) > 0, \forall s \in \mathcal{S}, i \in \mathcal{N}$.
- (ii) $V^\pi(s) = \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s), \forall s \in \mathcal{S}, i \in \mathcal{N}$.
- (iii) $V^\pi(s) = \max_{\pi'_i} \mathbb{E}_{\pi'_i, \pi_{-i}} [Q^\pi(s, a)], \forall s \in \mathcal{S}, i \in \mathcal{N}$.

Proof. (i) $\rightarrow$ (ii). Assume there exist $s \in \mathcal{S}, i \in \mathcal{N}$ such that $V^\pi(s) < \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s)$. Then, let π'' be a policy such that $V^{\pi''_i, \pi_{-i}}(s) = \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s), \forall s \in \mathcal{S}, i \in \mathcal{N}$. Thus, $\mathbb{E}_{s \sim \mu} [V^\pi(s)] < \mathbb{E}_{s \sim \mu} [V^{\pi''_i, \pi_{-i}}(s)] = \max_{\pi'_i} \mathbb{E}_{s \sim \mu} [V^{\pi'_i, \pi_{-i}}(s)]$, which contradicts (i).

(ii) $\rightarrow$ (i). This is straightforward by definition.

(ii) $\rightarrow$ (iii). It is clear that $V^\pi(s) \leq \max_{\pi'_i} \mathbb{E}_{\pi'_i, \pi_{-i}} [Q^\pi(s, a)]$. Consider the reverse inequality. For any $\pi'_i, \lim_{k \rightarrow \infty} T_{\pi'_i, \pi_{-i}}^k V^\pi(s) = V^{\pi'_i, \pi_{-i}}(s) \leq V^\pi(s)$. By the monotonicity of the Bellman operator [47], $\mathbb{E}_{\pi'_i, \pi_{-i}} [Q^\pi(s, a)] = T_{\pi'_i, \pi_{-i}} V^\pi(s) \leq V^\pi(s)$. Thus, $V^\pi(s) \geq \max_{\pi'_i} \mathbb{E}_{\pi'_i, \pi_{-i}} [Q^\pi(s, a)]$.

(iii) $\rightarrow$ (ii). Clearly, $V^\pi(s) \leq \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s)$. Consider the reverse inequality. For any $\pi'_i, T_{\pi'_i, \pi_{-i}} V^\pi(s) = \mathbb{E}_{\pi'_i, \pi_{-i}} [Q^\pi(s, a)] \leq V^\pi(s)$. By the monotonicity of the Bellman operator, $\lim_{k \rightarrow \infty} T_{\pi'_i, \pi_{-i}}^k V^\pi(s) = V^{\pi'_i, \pi_{-i}}(s) \leq V^\pi(s)$. Thus, $V^\pi(s) \geq \max_{\pi'_i} V^{\pi'_i, \pi_{-i}}(s)$. $\square$

F Construction of Action-Dependent Graphs

To elucidate the construction of ADGs, we present Algorithm 2 that efficiently derives an ADG from a given CG such that the condition (14) is satisfied.

G Experimental Details

Setup of Coordination Polymatrix Game. In matrix cooperative games, different coordination graphs and their corresponding action dependency graphs are shown in Figure 10. The payoff matrices in polymatrix coordination games are shown in Tables 1 to 14. Any payoff matrix not displayed in the tables is the same as that in Table 5.

Algorithm 2 Greedy Algorithm: Finding a Sparse ADG

Input: A CG G_c

Output: An ADG $G_d = (\mathcal{N}, \mathcal{E}_d)$

Initialize an empty graph $G_d = (\mathcal{N}, \mathcal{E}_d)$ with all vertices unindexed

for $i = 0$ **to** $n - 1$ **do**

 Assign index $n - i$ to a vertex among the unindexed ones, such that the size of $N_c((n - i)^{[+]})$ is minimized

end for

Construct the edge set $\mathcal{E}_d$ by adding edges (j, i) for each vertex $i \in \mathcal{N}$ and each $j \in N_c(i^{[+]})$ as specified in (14)

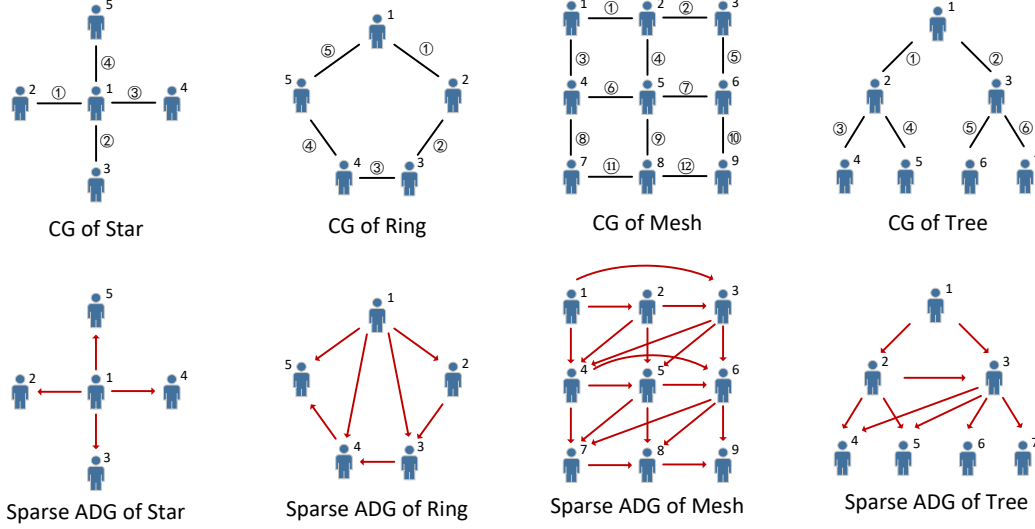


Figure 10: The CG and sparse ADG of polymatrix coordination game.

Table 1: ① in Star

$a_1 \backslash a_2$	0	1	2	3	4
0	3.5	5.0	0.5	0.5	0.5
1	0.5	3.5	6.0	0.5	0.5
2	0.5	0.5	3.5	0.5	0.5
3	0.5	0.5	0.5	3.25	0.5
4	0.5	0.5	0.5	0.5	3.0

Table 2: ② in Star

$a_1 \backslash a_3$	0	1	2	3	4
0	3.5	0.5	5.0	0.5	0.5
1	0.5	3.5	6.0	0.5	0.5
2	0.5	0.5	3.5	0.5	0.5
3	0.5	0.5	0.5	3.25	0.5
4	0.5	0.5	0.5	0.5	3.0

Table 3: ③ in Star

$a_1 \backslash a_4$	0	1	2	3	4
0	3.5	0.5	0.5	5.0	0.5
1	0.5	3.5	6.0	0.5	0.5
2	0.5	0.5	3.5	0.5	0.5
3	0.5	0.5	0.5	3.25	0.5
4	0.5	0.5	0.5	0.5	3.0

Table 4: ④ in Star

$a_1 \backslash a_5$	0	1	2	3	4
0	3.5	0.5	0.5	0.5	5.0
1	0.5	3.5	0.5	0.5	0.5
2	0.5	0.5	3.5	0.5	0.5
3	0.5	0.5	0.5	3.25	0.5
4	0.5	0.5	0.5	0.5	3.0

Experimental Hyperparameters. For algorithms that are not in tabular form, our implementation builds upon the open-source EPyMARL framework [51], employing the Adam optimizer for training. Unless otherwise specified, we adopt EPyMARL’s default hyperparameters, which are also detailed in the config of code provided in supplementary material. Table 15 lists the modified hyperparameters for different algorithms and environments.

Table 5: ① in Ring

$a_1 \backslash a_2$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	1.0	0.1	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 6: ② in Ring

$a_2 \backslash a_3$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	1.0	2.0	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 7: ③ in Ring

$a_3 \backslash a_4$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	1.0	0.1	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 8: ④ in Ring

$a_4 \backslash a_5$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	1.0	2.0	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 9: ⑤ in Ring

$a_1 \backslash a_5$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	10.0	0.1	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 10: ⑦ in Mesh

$a_5 \backslash a_6$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	1.0	2.0	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 11: ⑩ in Mesh

$a_6 \backslash a_9$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	10.0	0.1	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 12: ② in Tree

$a_1 \backslash a_3$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	2.0	1.5	0.1	0.1
2	0.1	0.1	1.0	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 13: ⑤ in Tree

$a_3 \backslash a_6$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	0.5	0.1	0.1	0.1
2	0.1	0.1	1.5	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

In Table 15, the target update interval is categorized into soft and hard updates: soft updates adjust the target network incrementally during each gradient descent step, while hard updates occur once every specified number of gradient descent steps. For the DCG algorithm, we utilize the default hyperparameters from [18]. Although we explored hyperparameter optimization for DCG, we found its performance sensitive to changes, exhibiting instability under alternative configurations.

Neural Network Architecture. For the MAPPO, QMIX, and DCG algorithms without ADGs, we adopt the default neural network configurations specified in [51] and [18]. For MAPPO and QMIX with ADGs, we only modify the agent network architecture as follows. Let $o_i \in \mathbb{R}^{d_o}$ denote the

Table 14: ⑥ in Tree

$a_3 \backslash a_7$	0	1	2	3	4
0	1.0	0.1	0.1	0.1	0.1
1	0.1	0.5	0.1	0.1	0.1
2	0.1	0.1	1.5	0.1	0.1
3	0.1	0.1	0.1	1.0	0.1
4	0.1	0.1	0.1	0.1	1.0

Table 15: Experimental Hyperparameters

	MAPPO(ATSC)	QMIX(ATSC)	DCG(ATSC)	QMIX(SMAC)	DCG(SMAC)
learning rate	0.0004	0.0001	0.0005	2^{-12}	0.0005
weight decay	0	0	0	2^{-11}	0
batch size	8	8	32	32	32
buffer size	8	32	500	4096	500
target update interval	0.01(soft)	0.01(soft)	100(hard)	200(hard)	200(hard)

observational features for agent i , and $a_{N_d(i)} \in \mathbb{R}^{d_a}$ represent the concatenated actions features of neighboring agents in the ADG. The output is computed as:

$$h_1 = \text{ReLU}(W_1 o_i + b_1), \quad h_2 = \text{ReLU}(W_2 h_1 + b_2), \quad z_o = W_3 h_2 + b_3, \quad (46)$$

where $W_1 \in \mathbb{R}^{64 \times d_o}$, $W_2 \in \mathbb{R}^{64 \times 64}$, $W_3 \in \mathbb{R}^{d_{\text{act}} \times 64}$, and $b_1, b_2 \in \mathbb{R}^{64}$, $b_3 \in \mathbb{R}^{d_{\text{act}}}$ are the weights and biases, respectively, mapping to a 64-dimensional vector at each hidden layer and an action-dimensional output $z_o \in \mathbb{R}^{d_{\text{act}}}$. Concurrently, the concatenated input $[o_i, a_{N_d(i)}] \in \mathbb{R}^{d_o + d_a}$ is processed through a parallel pipeline with distinct parameters:

$$h'_1 = \text{ReLU}(W'_1 [o_i, a_{N_d(i)}] + b'_1), \quad h'_2 = \text{ReLU}(W'_2 h'_1 + b'_2), \quad z_a = W'_3 h'_2 + b'_3, \quad (47)$$

where $W'_1 \in \mathbb{R}^{64 \times (d_o + d_a)}$, $W'_2 \in \mathbb{R}^{64 \times 64}$, $W'_3 \in \mathbb{R}^{d_{\text{act}} \times 64}$, and $b'_1, b'_2 \in \mathbb{R}^{64}$, $b'_3 \in \mathbb{R}^{d_{\text{act}}}$. The final output is obtained as:

$$z = z_o + z_a. \quad (48)$$

Setup of ATSC. The ATSC environment comprises nine agents. The CG and sparse ADG for ATSC are represented by adjacency lists, as shown in Table 16 and Table 17, respectively. Experiments are conducted using ten random seeds (1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000), with each run requiring approximately two hours on a system equipped with an Intel Xeon Platinum 8481C CPU @ 2.70GHz, 80GB of memory, and a GeForce RTX 4090 D GPU.

Table 16: CG of ATSC

Vertex	Neighbors
0	1, 3
1	0, 2, 4
2	1, 5
3	0, 4, 6
4	1, 3, 5, 7
5	2, 4, 8
6	3, 7
7	4, 6, 8
8	5, 7

Table 17: Sparse ADG of ATSC

Vertex	Out-Degree Neighbors
0	
1	0
2	1, 0
3	2, 1, 0
4	3, 2, 1
5	4, 3, 2
6	5, 4, 3
7	6, 5, 4
8	7, 5

Setup of SMAC. In the MMM2 map of the SMAC environment, our side controls ten agents (seven Marines, two Marauders, and one Medivac), while the opposing side commands 12 agents (eight Marines, three Marauders, and one Medivac). The artificial CG and sparse ADG are represented by

adjacency lists, as shown in Table 18 and Table 19, respectively. Experiments utilized ten random seeds (42, 126, 252, 420, 630, 882, 1176, 1512, 1890, 2310), with each run requiring approximately two hours on a system equipped with an Intel Xeon Platinum 8481C CPU @ 2.70GHz, 80GB of memory, and a GeForce RTX 4090 D GPU.

Table 18: CG of SMAC

Vertex	Neighbors
0	1, 3
1	0, 2, 4
2	1, 5
3	0, 4, 6
4	1, 3, 5, 7
5	2, 4, 8
6	3, 7, 9
7	4, 6, 8, 9
8	5, 7, 9
9	6, 7, 8

Table 19: Sparse ADG of SMAC

Vertex	Out-Degree Neighbors
0	
1	0
2	1, 0
3	2, 1, 0
4	3, 2, 1
5	4, 3, 2
6	5, 4, 3
7	6, 5, 4
8	5, 6, 7
9	6, 7, 8